

Disaster Recovery Guide

1. Recovery Objectives

Metric	Target	Rationale
RTO	≤ 2 hours	A single-VPS deployment re-provisioned via Terraform + cloud-init rebuilds the full stack well under this window
RPO	≤ 24 hours	Aligned with a daily Postgres/MinIO backup cadence

2. Data Classification

Data	Location	Impact	Recoverable elsewhere?
Indicators, assets, orgs, alerts, notices, events, knowledge chunks	PostgreSQL (postgres_data)	High	Knowledge chunks re-seedable; indicator/alert history cannot be
Evidence, reports, malware samples	MinIO (minio_data)	High	No — historical artifacts are lost
Search index	Elasticsearch (ephemeral)	Medium	Yes — fully rebuildable via backfill_search.py
Cache	Redis (ephemeral)	Low	Yes — self-heals on next threat check
Identity/realm config	Keycloak (backed by sop_db)	High	Recoverable only via Postgres backup
Case data	TheHive (in-container)	High	No — not currently backed up separately (known gap)
n8n workflows & credentials	n8n_data	Medium	Workflow JSON re-importable from Git; credentials must be re-entered
Suricata/Zeek/Falco raw logs	Named volumes	Low	Rolling logs, already summarized into Postgres
Infrastructure definition	Terraform state + Docker Compose	High	Already in Git — primary recovery path for the whole stack

3. Backup Strategy

3.1 PostgreSQL — daily logical backup

```
docker exec sop-postgres pg_dump -U sop_admin sop_db | gzip > \
/opt/soc-backups/sop_db_$(date +%Y%m%d).sql.gz
find /opt/soc-backups -name "sop_db_*.sql.gz" -mtime +14 -delete
```

Recommended: daily cron at a low-traffic hour, synced off-VPS (S3-compatible bucket or another Contabo box) so a full VPS loss doesn't also destroy backups.

3.2 MinIO — object storage mirror

```
mc alias set sop-local http://localhost:9000 sop_admin
mc mirror sop-local/reports /opt/soc-backups/minio/reports
mc mirror sop-local/evidence /opt/soc-backups/minio/evidence
```

3.3 Docker volumes — full snapshot (optional)

```
docker run --rm -v soc_n8n_data:/data -v /opt/soc-backups:/backup \
  alpine tar czf /backup/n8n_data_$(date +%Y%m%d).tar.gz -C /data .
# Repeat for grafana_data, wazuh_certs, wazuh_indexer_data
```

3.4 Infrastructure as code

terraform/ and docker-compose.yml are already in Git — the primary mechanism for rebuilding the entire stack topology from nothing.

4. Recovery Procedures

4.1 VPS destroyed / unrecoverable

- Provision a replacement via terraform apply — cloud-init installs Docker, clones the repo, runs docker compose up -d.
- docker compose down before it accepts traffic, to avoid writing to empty volumes.
- Restore latest Postgres dump: `gunzip -c sop_db_LATEST.sql.gz | docker exec -i sop-postgres psql -U sop_admin -d sop_db`
- Restore MinIO buckets via mc mirror in reverse; restore named-volume tarballs before first start or via docker cp.
- docker compose up -d, run the Deployment Guide §7 verification checklist.
- Re-enter any credentials not captured in volume backups (n8n IMAP/Slack/SMTP, if not restored via n8n_data).

4.2 PostgreSQL data corruption only

- docker compose stop backend keycloak (both depend on Postgres).
- Restore from the most recent pg_dump.
- docker compose start postgres backend keycloak; verify /health and Keycloak login.

4.3 A monitoring container fails (Suricata/Zeek/Falco/Wazuh)

The 60s watchdog raises a SystemAlert and Network Status shows it inactive. docker compose restart <service> — these are stateless collectors; log volumes persist independently. The next watchdog cycle re-ingests buffered data automatically.

4.4 TheHive data loss

No automated backup exists today (known gap). Recovery means manually re-creating the org and service-account API key; historical case notes/timelines are lost. New threat checks will re-create cases for currently suspicious/malicious IPs going forward.

5. Known Gaps & Recommendations

- TheHive has no backup path — add a scheduled export to /opt/soc-backups.
- Backups are local-to-VPS by default — sync to off-VPS storage on the same cron schedule.
- No automated restore test exists — recommend a quarterly fire-drill: restore the latest dump into a throwaway container and spot-check row counts.
- Elasticsearch and Redis are intentionally not backed up — both are fully rebuildable, by design.